

Documentation Technique

Script PowerShell – Collecte de fichiers .LOG multiserveurs

PYRCZAK Noah | CIMPA – Sopra Steria | Stage Semaine 1

1. Contexte et objectif

Lors de la première semaine de stage chez CIMPA – Sopra Steria, j'ai eu pour mission de développer un script PowerShell permettant de centraliser automatiquement les fichiers de logs produits par l'application TeamCenter sur plusieurs serveurs distants.

L'application TeamCenter génère des fichiers .LOG sur différents serveurs (TC Server, Apache, Tomcat). Ces logs étaient jusqu'alors consultés manuellement sur chaque machine. L'objectif était d'automatiser leur collecte, leur centralisation et leur archivage.

Les objectifs du script sont :

- Parcourir automatiquement les chemins de logs sur plusieurs serveurs distants
- Copier tous les fichiers .LOG dans un dossier centralisé sur le Bureau de l'utilisateur
- Créer une archive ZIP horodatée pour archivage
- Gérer les erreurs d'accès réseau sans interrompre le script

2. Serveurs et chemins surveillés

Le script prend en charge trois chemins de logs correspondant aux serveurs utilisés dans l'infrastructure TeamCenter :

Elément	Explication
TC Server	Serveur principal TeamCenter – logs applicatifs (chemin : \\IPDuServeurTC\C\$\Temp\Logs)
Apache (Web Server)	Serveur web Apache – logs HTTP et accès (chemin : \\IPDuServeurWeb\C\$\Apache\Logs)
Tomcat (Web Server)	Serveur d'applications Tomcat – logs Java/servlet (chemin : \\IPDuServeurWeb\C\$\Tomcat\Logs)

Note : lors du développement, les adresses IP exactes des serveurs n'étaient pas disponibles. Des chemins fictifs ont été utilisés pour la structure du script, à remplacer lors du déploiement réel.

3. Structure et explication du script

3.1 Configuration des chemins et initialisation

Le script commence par déclarer un tableau contenant les chemins UNC des dossiers de logs sur chaque serveur, puis prépare le dossier de destination sur le Bureau.

Code – Configuration initiale

```
$cheminsLogsServeurs = @(
    "\\IPDuServeurTC\C$\Temp\Logs",
    "\\IPDuServeurWeb\C$\Apache\Logs",
    "\\IPDuServeurWeb\C$\Tomcat\Logs"
)
$cheminBureau = [Environment]::GetFolderPath("Desktop")
$dossierDestination = Join-Path -Path $cheminBureau -ChildPath "LogsCollectes"
```

Element	Explication
<code>\$cheminsLogsServeurs</code>	Tableau (array) des chemins UNC vers les dossiers de logs sur chaque serveur distant
<code>[Environment]::GetFolderPath</code>	Methode .NET qui retourne le chemin du Bureau de l'utilisateur actif, de façon dynamique
<code>Join-Path</code>	Combine proprement deux chemins sans risque de double séparateur
<code>\$dossierDestination</code>	Chemin du dossier 'LogsCollectes' qui sera créé sur le Bureau

3.2 Création du dossier de destination

Avant de copier les fichiers, le script vérifie si le dossier de destination existe. S'il n'existe pas, il le crée automatiquement.

Code – Création du dossier

```
if (-not (Test-Path $dossierDestination)) {
    New-Item -ItemType Directory -Path $dossierDestination | Out-Null
    Write-Host "Dossier créé : $dossierDestination"
} else {
    Write-Host "Le dossier existe déjà : $dossierDestination"
}
```

Element	Explication
<code>Test-Path</code>	Vérifie si un chemin (fichier ou dossier) existe sur le système de fichiers
<code>-not</code>	Operateur de négation : le bloc s'exécute uniquement si le dossier n'existe PAS
<code>New-Item -ItemType Directory</code>	Crée un nouveau dossier au chemin spécifié
<code> Out-Null</code>	Supprime l'affichage de la sortie de New-Item pour garder la console propre

3.3 Boucle de collecte des logs – foreach

Le cœur du script est une boucle foreach qui parcourt chaque chemin serveur. Pour chaque serveur, il vérifie l'accessibilité du chemin, puis copie tous les fichiers .LOG trouve.

Code – Boucle de collecte

```
foreach ($cheminLog in $cheminsLogsServeurs) {  
    if (Test-Path $cheminLog) {  
        try {  
            Get-ChildItem -Path $cheminLog -Recurse -Include *.log | ForEach-Object {  
                Copy-Item -Path $_.FullName -Destination $dossierDestination -Force  
            }  
        }  
        catch {  
            Write-Warning "Echec copie depuis $cheminLog : $($_.Exception.Message)"  
        }  
    } else {  
        Write-Warning "Chemin inaccessible : $cheminLog"  
    }  
}
```

Élément	Explication
<code>foreach</code>	Itère sur chaque chemin de la liste \$cheminsLogsServeurs
<code>Test-Path \$cheminLog</code>	Vérifie que le chemin réseau est accessible avant de tenter la copie
<code>Get-ChildItem -Recurse</code>	Liste tous les fichiers de façon récursive dans le dossier et ses sous-dossiers
<code>-Include *.log</code>	Filtre pour ne récupérer que les fichiers avec l'extension .log
<code>\$_ .FullName</code>	Variable automatique PowerShell : chemin complet du fichier en cours de traitement
<code>Copy-Item -Force</code>	Copie le fichier en écrasant s'il existe déjà dans la destination
<code>Write-Warning</code>	Affiche un message d'avertissement (en jaune) sans interrompre le script

3.4 Création de l'archive ZIP

Une fois tous les logs collectés, le script compresse le dossier en une archive ZIP horodatée pour faciliter l'archivage et le transfert.

Code – Archive ZIP

```
$nomFichierZip = "LogsCollectes_" + (Get-Date -Format "yyyyMMdd_HH:mm:ss") + ".zip"  
$cheminFichierZip = Join-Path -Path $cheminBureau -ChildPath $nomFichierZip  
  
try {  
    Compress-Archive -Path "$dossierDestination\*" -DestinationPath $cheminFichierZip -Force  
    Write-Host "Archive ZIP créée : $cheminFichierZip"  
}
```

```
catch {
    Write-Warning "Echec creation ZIP : $($_.Exception.Message)"
}
```

Elément	Explication
<code>Get-Date -Format 'yyyyMMdd_HH:mm:ss'</code>	Génère un horodatage unique pour nommer le fichier ZIP sans conflit
<code>Compress-Archive</code>	Cmdlet PowerShell natif pour créer une archive ZIP à partir d'un dossier ou fichier
<code>-Path "\$dossierDestination*"</code>	Le wildcard * sélectionne le contenu du dossier (pas le dossier lui-même)
<code>-DestinationPath</code>	Chemin complet du fichier ZIP à créer
<code>-Force</code>	Ecrase le ZIP existant s'il porte le même nom

3.5 Gestion des erreurs réseau (WinRM / accès distants)

Une difficulté majeure de la semaine a été l'impossibilité d'accéder aux serveurs distants via PowerShell à cause de la configuration WinRM et des droits d'accès. Le script intègre une gestion défensive de ces cas.

Commandes testées pour l'accès distant

```
# Tester la connexion à distance
Enter-PSSession -ComputerName IPDuServeur -Credential (Get-Credential)

# Exécuter une commande à distance
Invoke-Command -ComputerName IPDuServeur -ScriptBlock { Get-Service }

# Vérifier la configuration WinRM
Test-WSMan -ComputerName IPDuServeur
```

Elément	Explication
<code>Enter-PSSession</code>	Ouvre une session PowerShell interactive sur un ordinateur distant
<code>Invoke-Command</code>	Exécute un bloc de script sur un ou plusieurs ordinateurs distants
<code>Get-Credential</code>	Ouvre une boîte de dialogue pour saisir les identifiants de connexion
<code>Test-WSMan</code>	Vérifie si WinRM (Windows Remote Management) est actif sur le serveur cible
<code>WinRM</code>	Protocole Windows permettant l'exécution de commandes PowerShell à distance (port 5985/5986)

4. Compétences Bloc 1

Les indicateurs de performance ci-dessous sont repris mot pour mot du référentiel BTS SIO – Annexe I.B, Bloc 1.

4.1 Gérer le patrimoine informatique

Code	Sous-compétence	Indicateur de performance (référentiel)
Recenser et identifier les ressources numériques	Recenser et identifier les ressources numériques	<i>Le recensement du patrimoine informatique est exhaustif et réalise au moyen d'un outil de gestion des actifs informatiques.</i>
Exploiter des référentiels, normes et standards	Exploiter des référentiels, normes et standards adoptés par le prestataire informatique	<i>Les référentiels, normes et standards sont mobilisés de façon pertinente.</i>
Vérifier les conditions de la continuité d'un service	Vérifier les conditions de la continuité d'un service informatique	<i>Les conditions de continuité et de reprise d'un service sont vérifiées et les manquements sont signalés.</i>
Gérer des sauvegardes	Gérer des sauvegardes	<i>Les sauvegardes sont réalisées dans les conditions prévues conformément au plan de sauvegarde. Les restaurations sont testées et opérationnelles.</i>
Vérifier le respect des règles d'utilisation	Vérifier le respect des règles d'utilisation des ressources numériques	<i>Les écarts par rapport aux règles d'utilisation des ressources numériques sont détectés et signalés.</i>
Collecter, suivre et orienter des demandes	Collecter, suivre et orienter des demandes	<i>Le compte rendu d'intervention est clair et explicite. La communication écrite et orale est adaptée à l'interlocuteur.</i>
Traiter des demandes concernant les services réseau et système	Traiter des demandes concernant les services réseau et système, applicatifs	<i>La méthode de diagnostic de résolution d'un incident est adéquate et efficiente. Une solution à l'incident est trouvée et mise en œuvre.</i>
Traiter des demandes concernant les services réseau et système	Traiter des demandes concernant les services réseau et système, applicatifs	<i>La méthode de diagnostic de résolution d'un incident est adéquate et efficiente. Une solution à l'incident est trouvée et mise en œuvre.</i>
Analyser les objectifs et les modalités d'organisation	Analyser les objectifs et les modalités d'organisation d'un projet	<i>Les objectifs et les modalités d'organisation du projet sont explicites. L'analyse des besoins et de l'existant est pertinente.</i>
Planifier les activités	Planifier les activités	<i>Les activités personnelles sont planifiées selon une méthodologie donnée. Le découpage en tâches est réaliste. Les livrables sont conformes.</i>

Réaliser les tests d'intégration et d'acceptation	Réaliser les tests d'intégration et d'acceptation d'un service	<i>Des tests pertinents d'intégration et d'acceptation sont rédigés et effectués. Les outils de test sont utilisés de manière appropriée. Un rapport de test du service est produit.</i>
Déployer un service	Déployer un service	<i>Le service déployé est opérationnel et donne satisfaction à l'utilisateur.</i>
Accompagner les utilisateurs	Accompagner les utilisateurs dans la mise en place d'un service	<i>Un support d'information est disponible. Les modalités d'accompagnement sont définies.</i>

Déployer un service	Déployer un service	<i>Le service déployé est opérationnel et donne satisfaction à l'utilisateur.</i>
Accompagner les utilisateurs	Accompagner les utilisateurs dans la mise en place d'un service	<i>Un support d'information est disponible. Les modalités d'accompagnement sont définies.</i>
Mettre en œuvre des outils et stratégies de veille	Mettre en œuvre des outils et stratégies de veille informationnelle	<i>La veille est régulière et vise à utiliser de manière approfondie des moyens de recherche d'information et à renforcer ses compétences.</i>

5. Difficultés rencontrées et solutions

Problème rencontré	Solution apportée
Impossibilité d'accéder aux serveurs distants via PowerShell (droits Win RM non configurés)	Lecture de la documentation Microsoft sur Enter-Possession et Invoke-Command, tests locaux, puis correction de la configuration réseau avec le tuteur.
Adresses IP des serveurs inconnues lors du développement (TeamCenter, Apache, Tomcat)	Utilisation de chemins fictifs temporaires pour la structure du script, à remplacer lors du déploiement réel.
Risque d'écrasement de fichiers portant le même nom depuis différents serveurs	Ajout du paramètre -Force sur Copy-Item et horodatage du ZIP pour garantir l'unicité de chaque archive générée.